Kayden Conner

Worksheet 05 Answer Sheet

part 1

Q1.

1. Base, Derived
2. is
3. inherits / derives

Q2.

1. make
2. model
3. topSpeed
4. Describe()

part 2 Q3

ACCESS SPECIFIER	VISIBLE INSIDE THE SAME CLASS	VISIBLE IN A DERIVED CLASS	VISIBLE OUTSIDE (MAIN())
Private	Y	N	N
Protected	Y	Y	N
Public	Y	Y	Y

part 3 Q4, Q5

Q4: Prints A B C - construction always runs base-first, all the way down the chain, before the most-derived constructor body finishes. Destructors run in the exact opposite order: ~C ~B ~A

Q5:

```
Motorcycle::Motorcycle(string mk, string mo, int speed, bool sidecar)
: Vehicle(mk, mo, speed), hasSidecar(sidecar) {}
```

Part 4 Q6, Q7

Q6: v.describe(); prints "A vehicle." -- c.describe(); prints "A car." To call the base version from inside the override: Vehicle::describe();

1. Car(string) / Car(string,int) - **OVERLOADED** (same name, different parameter lists, same class)
2. Vehicle::describe() / Car::describe() - **OVERRIDDEN** (identical signature, redefined in the derived class)
3. Vehicle::describe() / Car::honk() - **NEITHER** (different names - unrelated functions)
4. operator+(Fraction) / operator+(int) - **OVERLOADED**

Part 5 Q8

```
class Motorcycle : public Vehicle {
    bool hasSidecar;
    public:
        Motorcycle(string mk, string mo, int spd, bool sidecar)
            : Vehicle(mk, mo, spd), hasSidecar(sidecar) {}

            void describe() {
                Vehicle::describe();
                cout << (hasSidecar ? "Has a sidecar.\n" : "No sidecar.\n");
            }
};
```

Part 6 Case A, B, C

Find the Error

Case A:

[class Car : Vehicle] with no keyword defaults to **private** inheritance (mirrors W01: no label = private). That turns every inherited **public** member of **Vehicle**, including **describe()**, into a **private** member of **Car**. Line X ([c.describe();] in **main()**) does ***NOT compile*** - **describe** is private within this context. Fix: [class Car : public Vehicle].

Case B:

Does ***NOT compile***. [Car(int doors)] never chains to a **Vehicle** constructor, so the compiler tries to call supply **mk**. Fix: give **Car** a **string** parameter too and chain it -- **Car(string mk, int doors) : Vehicle(mk), numDoors(doors) {}**.

Case C:

Does ***NOT compile***. **protected** only widens access for derived classes - it does nothing for outside code. **main()** is not a member of **Vehicle** or any class derived from it, so **v.make** is just as unreachable as if **make** were **private**.